Cryptocurrency Volatility Prediction Using Multimodal Deep Learning: Integrating Market Time-Series and Twitter Sentiment

Ayush Bharatbhai Patel
MSc Data Science
University of Roehampton, London
Email: patela36@roehampton.ac.uk
Project Repository Link : https://github.com/patela2151-blip/crypto-volatility-multimodal-dl

***Abstract*—Cryptocurrency markets exhibit heavy-tailed returns, volatility clustering, abrupt regime shifts, and rapid information diffusion, making volatility forecasting a difficult non-stationary prediction task. This paper presents a leakage-safe multimodal deep learning framework for forecasting Bitcoin realised volatility by combining structured market time-series features with Twitter-derived sentiment features. Tweet-level sentiment is extracted using a transformer-based classifier, aggregated into daily sentiment descriptors, and fused with market variables before being passed to a Long Short-Term Memory (LSTM) regression model trained on fixed-length chronological windows.**

A key contribution of the study is not only the forecasting model itself, but the practical engineering strategy used to make the full multimodal pipeline executable under CPU-only constraints. Instead of performing transformer inference over the full raw tweet corpus, the final system samples up to 100 tweets per day and performs batched sentiment inference, reducing the processed text workload from approximately 170,000 raw tweets to 900 sampled tweets while preserving daily sentiment aggregation.

In the final aligned evaluation, the full multimodal LSTM was assessed on 125 aligned test observations and achieved an MSE of 2.6496×10^{-5}, an RMSE of 0.005147, and an MAE of 0.003505. The persistence baseline remained stronger with RMSE 0.004531, while the market-only LSTM achieved RMSE 0.005099. Diagnostic analysis shows stable optimisation, non-flat forecasts, near-zero-centred residuals, and time-varying error across the test window. The study therefore contributes a reproducible multimodal forecasting pipeline, a realistic implementation strategy for limited-hardware environments, and an honest empirical finding: sentiment-enhanced deep learning is technically viable for Bitcoin volatility forecasting, but it does not automatically outperform strong simpler baselines when sentiment coverage is sparse.

***Index Terms*—Cryptocurrency, volatility forecasting, multimodal learning, LSTM, transformer, sentiment analysis, time-series regression, Bitcoin**

I. Introduction

Cryptocurrency markets provide a challenging environment for predictive modelling because they are highly volatile, weakly regulated compared with mature financial markets, and very sensitive to rapidly spreading information. Bitcoin, in particular, is traded continuously and responds quickly to macroeconomic announcements, exchange-level events, social media narratives, and speculative trading behaviour. For this reason, the problem addressed in this project is not simply whether the next day will close higher or lower, but how unstable the market is likely to be. This is a more meaningful and more difficult task because volatility is directly related to risk, uncertainty, and the reliability of trading or hedging decisions.

Volatility forecasting matters in practice because it influences risk management, leverage control, position sizing, derivative pricing, and stress testing. A trader or portfolio manager can tolerate being wrong about small day-to-day directional movements more easily than being unprepared for a sudden expansion in volatility. In the cryptocurrency setting, where large swings can occur over short periods and market behaviour is strongly regime-dependent, this forecasting task becomes especially important.

A further difficulty is that Bitcoin volatility is not driven only by its own recent price history. Cryptocurrency markets are deeply affected by online narrative formation. Tweets, community discussions, and sudden bursts of collective attention often amplify price reactions and uncertainty. A model that uses only structured market variables may therefore learn the dominant persistence pattern in realised volatility but fail to capture part of the external sentiment environment that accompanies large shifts in market behaviour. This motivates the use of multimodal learning, in which numerical time-series features and text-derived sentiment features are integrated into a single forecasting framework.

The aim of this work is to build and evaluate a multimodal deep learning model that predicts daily Bitcoin realised volatility using both market variables and Twitter sentiment. The chosen architecture combines transformer-based sentiment inference at the text stage with an LSTM-based sequence regressor at the forecasting stage. The project also places considerable emphasis on implementation realism. In coursework settings, one of the biggest gaps between an attractive idea and

a successful submission is whether the full system can actually be executed, diagnosed, and evaluated under the available compute constraints. That is why a substantial part of this work focuses on robust preprocessing, leakage-safe temporal alignment, and a sentiment-processing strategy that is feasible on CPU.

The main contributions of the project can be summarised as follows:

- a clearly defined and non-trivial forecasting problem focused on Bitcoin realised volatility rather than simple price-direction classification;
- a multimodal deep learning solution that combines structured market history with transformer-derived sentiment signals;
- a robust preprocessing pipeline capable of tolerating malformed tweet rows, inconsistent timestamp formats, and missing sentiment observations;
- a CPU-feasible sentiment extraction workflow based on daily sampling and batched inference;
- a leakage-safe chronological evaluation design with clear baseline and ablation comparisons; and
- a critical interpretation of results showing where model complexity helps, where it does not, and why strong baselines remain difficult to beat.

II. Problem Definition and Solution Selection

The selected problem is daily Bitcoin volatility prediction. This problem is substantially more challenging than next-day movement classification because volatility is noisy, clustered, and regime-dependent. It is also more relevant from a quantitative-finance perspective. While direction can matter for speculation, volatility is central to uncertainty estimation and risk control. In a market such as Bitcoin, where structural breaks and abrupt spikes are common, a useful volatility model can provide more practical value than a simple up/down predictor.

The solution was designed around deep learning because the underlying mapping is both sequential and non-linear. Recent market behaviour matters, but it matters in a state-dependent way. The effect of a return pattern today may differ depending on whether the market is already in a high-volatility regime, whether sentiment is unusually negative, or whether the previous week has been calm. An ordinary linear model would struggle to represent this combination adequately. An LSTM was selected as the main predictive model because it is well suited to sequence learning, has a strong record in financial time-series applications, and is appropriate for modest dataset sizes compared with more parameter-heavy transformer-only forecasting architectures.

The text side of the solution also required an explicit design choice. Tweets are short, informal, and often ambiguous in isolation. Transformer-based models are therefore better suited to sentiment extraction than simple polarity dictionaries or bag-of-words approaches. However, running transformer inference over a very large raw tweet corpus can easily become computationally prohibitive. For that reason, the final solution balances modelling ambition and computational realism: transformer sentiment is used, but with sampling and batching so that the complete pipeline remains runnable and reproducible.

III. Background and Literature Review

Volatility forecasting has long been a central topic in financial econometrics because asset returns rarely exhibit constant variance. Instead, they display heteroskedasticity and volatility clustering, where turbulent periods tend to be followed by further turbulence and calm periods tend to persist. Classical econometric models such as ARCH and GARCH were developed specifically to represent this behaviour and remain important reference points because of their interpretability and explicit treatment of conditional variance [1]–[3]. However, cryptocurrency markets pose additional challenges for these models because return distributions are often heavy-tailed, market structure changes rapidly, and narrative-driven regime shifts can occur abruptly.

Deep learning methods have been proposed as flexible alternatives because they can learn non-linear temporal relationships directly from data without imposing a rigid variance specification. In particular, Long Short-Term Memory (LSTM) networks are designed to preserve information across multiple time steps through gated memory mechanisms, making them suitable for financial time-series problems in which recent history matters but the relevant dependency horizon is not fixed in advance [4], [6], [13]. At the same time, prior work also shows that deep learning should not be treated as automatically superior in financial prediction tasks. In noisy short-horizon settings with limited effective sample size, complex neural models may offer only modest gains over simpler baselines or may fail to improve on them at all [6], [13].

A separate stream of research investigates sentiment as an auxiliary predictive signal. Social-media and news-based studies argue that public mood, uncertainty, and attention surges may influence or anticipate market behaviour [7], [8]. Recent advances in transformer-based language models, particularly BERT-style architectures, have improved sentiment extraction by capturing contextual meaning rather than relying on isolated keywords or dictionary scores [5], [10]. This is especially relevant in cryptocurrency discourse, where tweets are short, informal, and often shaped by hype, irony, or rapidly shifting reactions. However, the usefulness of sentiment in downstream forecasting does not depend only on the text classifier itself. It also depends on whether sentiment observations are temporally aligned with the target market series, whether coverage is dense enough to be informative, and whether daily aggregation preserves useful structure rather than washing it out.

Multimodal financial forecasting attempts to integrate these complementary sources of information by combining structured market features with lower-dimensional summaries of unstructured text [12]. The theoretical motivation is appealing: prices reflect realised market behaviour, whereas text may capture emerging narratives before they are fully absorbed into observable price dynamics. Nevertheless, empirical gains from multimodal fusion are highly context dependent. When

the auxiliary text modality is sparse, weakly aligned, or dominated by noise, it can add complexity without improving predictive accuracy. This study is therefore positioned not as an attempt to prove that multimodal deep learning is always better, but as a careful empirical test of whether a sentiment-enhanced LSTM provides measurable benefit for Bitcoin realised-volatility forecasting under realistic computational constraints.

IV. Data Acquisition, Exploration, and Preprocessing

A. Bitcoin market data

The structured numerical component of the project uses daily Bitcoin market data for the ticker `BTC-USD`, downloaded through the `yfinance` library over the period from **1 January 2021** to **31 December 2023**. Rather than forecasting raw price directly, the market series was transformed into a volatility target that reflects short-term risk.

Closing prices were first converted into daily log returns:

$$r_t = \log(P_t) - \log(P_{t-1}), \tag{1}$$

where P_t denotes the closing price on day t.

Daily realised volatility was then computed as a rolling standard deviation of returns over a fixed **7-day** window:

$$\sigma_t = \sqrt{\frac{1}{W-1}\sum_{i=0}^{W-1}(r_{t-i} - \bar{r}_t)^2}, \tag{2}$$

where $W = 7$. This transforms the raw market series into a direct measure of short-term variability, which is more appropriate for the forecasting objective of this study than simple price-direction prediction.

After return construction and volatility estimation, the processed market dataset contained **1087 usable BTC rows**. Following multimodal merging, missing-value handling, and lag creation, the final base dataset contained **1080 rows** before chronological splitting and sequence generation. Before modelling, the market data were also checked for missing values, invalid return calculations, and rows affected by rolling-window edge effects. Since realised volatility depends on a historical window, the earliest observations could not support target construction and were removed only after the derived variables had been created, which avoided unnecessary loss of usable data.

B. Twitter data and ingestion challenges

The unstructured side of the project consists of Bitcoin-related tweets. In theory, this modality is attractive because cryptocurrency markets are heavily influenced by narrative, attention, and social mood. In practice, however, tweet data are much messier than structured financial tables. The raw tweet corpus contained approximately 170,000 records and exhibited common real-world data problems such as malformed rows, inconsistent timestamp formats, missing text, embedded line breaks, and noisy formatting. If not handled carefully, these issues can cause parsing failures, incorrect temporal alignment, or silent data loss.

To address this, the ingestion stage was deliberately designed to be fault tolerant. Timestamps were parsed using error coercion so that unusable rows could be detected rather than crashing the pipeline. Invalid rows were removed only where necessary, and text fields were normalised into a consistent format before sentiment inference. This design choice is important because in a multimodal setting, fragility in one modality can undermine the entire system.

C. Sampling and computational feasibility

One of the most practically important implementation decisions in this project concerns the cost of transformer inference. Applying a transformer-based sentiment model to all 170,000 raw tweets on CPU would have made the pipeline too slow for repeated experimentation, debugging, and fair comparison of model variants. This would have weakened the project, because a forecasting system that cannot be run consistently is difficult to evaluate honestly.

To make the pipeline executable on CPU, the implementation samples up to 100 tweets per day before sentiment inference. Sampling is performed with a fixed random seed to improve reproducibility. In the final run, this reduced the sentiment-processing workload to 900 tweets, which made it feasible to execute the full multimodal pipeline repeatedly for debugging, ablation testing, and fair model comparison.

D. Sentiment extraction and daily aggregation

Tweet-level sentiment was extracted using the transformer model cardiffnlp/twitter-roberta-base-sentiment. Inference was performed in batches of 32 with a maximum token length of 128. For each tweet, the model outputs probabilities for negative, neutral, and positive sentiment, and a continuous sentiment score was computed as the positive probability minus the negative probability. These tweet-level scores were then aggregated by day into mean sentiment, sentiment standard deviation, and tweet volume. For a given tweet x, the classifier returns

$$p(x) = [p_{\text{neg}}(x), p_{\text{neu}}(x), p_{\text{pos}}(x)]. \tag{3}$$

Rather than keeping only the most likely class, the implementation defines a continuous sentiment score as

$$s(x) = p_{\text{pos}}(x) - p_{\text{neg}}(x). \tag{4}$$

This retains both polarity and intensity and is more useful for regression-oriented downstream modelling than a hard sentiment label.

Tweet-level sentiment scores were then aggregated at the daily level into three descriptors: mean sentiment, sentiment dispersion, and tweet volume. These daily summaries are compact enough to be fused with market variables while still preserving useful information about overall tone, disagreement, and attention. A major limitation, however, is that only 9 sentiment-bearing days remained in the final aligned run. Missing sentiment values were therefore filled with neutral

defaults so that the market time series remained continuous rather than fragmenting around sparse text observations.

V. Model Development and Implementation Details

A. Target transformation

The forecasting target in this study is daily realised volatility derived from Bitcoin log returns. Because realised volatility is strictly positive and usually right-skewed, the main experiment applies a logarithmic transformation to stabilise optimisation and reduce the influence of extreme spikes:

$$y_t = \log(\sigma_t + \epsilon), \tag{5}$$

where σ_t denotes realised volatility on day t and ϵ is a small constant added for numerical stability. In the implemented pipeline, $\epsilon = 10^{-8}$. A separate ablation experiment was also conducted without this transformation in order to test whether the log scale was materially responsible for model behaviour.

B. Feature engineering and multimodal fusion

For each trading day, a fused multimodal feature vector was constructed by combining market variables, text-derived sentiment summaries, and lagged volatility features:

$$\mathbf{z}_t = [P_t,\ r_t,\ \mu_t,\ \sigma_t^{(s)},\ v_t,\ \sigma_{t-1},\ \sigma_{t-7}], \tag{6}$$

where P_t is the Bitcoin closing price, r_t is the daily log return, μ_t is the daily mean sentiment, $\sigma_t^{(s)}$ is the daily sentiment standard deviation, v_t is the daily tweet volume, and σ_{t-1} and σ_{t-7} are one-day and seven-day lagged realised volatility values.

The market data were downloaded for BTC–USD over the period from 1 January 2021 to 31 December 2023. Daily returns were computed as log differences of closing prices, and realised volatility was defined as a 7-day rolling standard deviation of returns. On the text side, Bitcoin-related tweets were cleaned and grouped by date. To keep the pipeline feasible on CPU, the implementation sampled up to 100 tweets per day before sentiment inference. Tweet-level sentiment was then extracted using the `cardiffnlp/twitter-roberta-base-sentiment` transformer model. Rather than using only a hard sentiment class, the system computed a continuous sentiment score from model probabilities as the positive probability minus the negative probability. These tweet-level scores were aggregated into daily sentiment mean, daily sentiment dispersion, and daily tweet volume.

After merging the market and sentiment modalities by date, missing sentiment values were filled with neutral defaults so that the market time series remained continuous even when no tweet-derived sentiment observations were available. The lagged volatility variables were then added explicitly because volatility persistence is known to be strong, and these hand-crafted temporal cues complement the sequential modelling capacity of the LSTM.

C. Leakage-safe preprocessing

All features were standardised using statistics estimated from the training split only. This is an important design choice in time-series forecasting because fitting scalers on the full dataset would leak future information into the model-development process. Separate standardisation was also applied to the target variable, again using the training partition only, after which the validation and test partitions were transformed using the already-fitted scalers.

D. Sequence construction

The fused daily feature vectors were converted into overlapping fixed-length sequences using a sliding-window procedure:

$$\mathbf{X}_t = [\mathbf{z}_{t-L+1}, \ldots, \mathbf{z}_t] \in \mathbb{R}^{L \times D}, \tag{7}$$

where L is the sequence length and D is the feature dimension. In the final implementation, $L = 30$, meaning that the model uses the previous 30 daily observations to predict the next volatility target. This window length was selected as a practical compromise: it is long enough to capture short-term clustering and regime persistence, but not so long that the number of usable sequences becomes unnecessarily small.

E. LSTM regressor

The forecasting model is a compact PyTorch LSTM regressor. The network contains two recurrent layers with hidden size 64 and dropout 0.3 applied between recurrent layers. Given an input sequence $\mathbf{X}_t$, the LSTM processes the sequence chronologically and produces a hidden representation at each time step. The hidden state from the final time step is then passed through a single linear output layer to produce a scalar volatility forecast:

$$\hat{y}_t = f_\theta(\mathbf{X}_t), \tag{8}$$

where f_θ denotes the learned mapping parameterised by the network weights.

The model was intentionally kept compact. Financial time-series datasets are often noisy and comparatively small, and unnecessarily large architectures can increase variance without improving generalisation. A lighter recurrent model was therefore more appropriate for this coursework setting than a more complex forecasting architecture.

F. Loss function, optimisation, and training control

Model parameters were learned using Smooth L1 loss:

$$\mathcal{L}(y_t, \hat{y}_t) = \begin{cases} \frac{1}{2}(y_t - \hat{y}_t)^2, & |y_t - \hat{y}_t| < \beta, \\ \beta|y_t - \hat{y}_t| - \frac{1}{2}\beta^2, & \text{otherwise}, \end{cases} \tag{9}$$

with $\beta = 0.01$. This loss was preferred over plain mean squared error because it is more robust to occasional large deviations, which are common in volatility forecasting.

Optimisation used the Adam optimiser with learning rate 0.001, batch size 32, and no additional weight decay. Training was run for a maximum of 60 epochs, but model selection was governed by validation performance rather than by the final epoch alone. A ReduceLROnPlateau scheduler was used

to reduce the learning rate by a factor of 0.5 when validation loss stopped improving, with scheduler patience of 3 epochs. In addition, early stopping was applied with patience 8 and minimum improvement threshold 10^{-4}. Whenever validation loss improved sufficiently, the current model parameters were saved; otherwise the remaining patience budget was reduced. This design reduces unnecessary overtraining and helps retain the best-performing model checkpoint.

To improve reproducibility, all random seeds were fixed and the training data loader was kept non-shuffled so that temporal ordering remained intact throughout the experimental process. The implementation also stores result summaries, figures, and best model checkpoints in separate output directories, making reruns and inspection easier.

VI. Experimental Setup and Evaluation Design

A. Chronological data split

Evaluation was conducted using a strict chronological split rather than random shuffling. Let N denote the total number of usable daily observations after preprocessing and target construction. The dataset was divided in time order into:

- 70% training data,
- 15% validation data,
- 15% test data.

This split reflects the real forecasting setting more faithfully because future observations are not allowed to influence model development. Random shuffling was deliberately avoided, since it would mix past and future regimes and create an unrealistically optimistic estimate of predictive performance.

B. Main experiment and ablation settings

The main experiment, labelled `multimodal_full`, used the complete fused feature set together with the log-transformed target. Several comparison variants were then evaluated under the same preprocessing and split logic:

- `market_only`: removes sentiment features while retaining market and lagged-volatility inputs;
- `no_lag_features`: removes lagged volatility features while retaining price, return, and sentiment summaries;
- `price_return_only`: uses only the most basic market variables;
- `no_log_target_full`: keeps the full feature set but predicts volatility on the original scale rather than the logarithmic scale.

This experimental design makes it possible to assess not only the total performance of the proposed model, but also which components of the multimodal pipeline contribute most to final behaviour.

C. Evaluation metrics

Model quality was assessed using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE):

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \tag{10}$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \tag{11}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{12}$$

These three metrics were chosen because they capture complementary aspects of forecasting error. RMSE is especially useful in this application because it penalises large misses more heavily, which matters when high-volatility periods are the most important events to anticipate.

D. Baseline models and aligned comparison

To place the LSTM results in context, three simple but strong baselines were evaluated on the same test horizon:

- a persistence baseline, $\hat{\sigma}_{t+1} = \sigma_t$;
- a 7-day rolling mean baseline;
- an exponentially weighted moving average (EWMA) baseline with $\alpha = 0.94$.

Because some baseline forecasts are undefined at the earliest test positions, final comparison was performed only on the aligned region where all baseline predictions were available simultaneously. This fair-alignment procedure ensures that the proposed model and all baselines are compared on exactly the same subset of observations rather than on slightly different evaluation windows.

E. Bootstrap confidence intervals and diagnostics

In addition to point estimates of performance, the implementation supports bootstrap confidence intervals for selected metrics. When enabled, 1000 bootstrap resamples are drawn from the aligned evaluation set in order to estimate uncertainty ranges for RMSE and MAE. This does not change the main ranking of models, but it provides additional information about the stability of reported errors.

For the main multimodal experiment, a set of diagnostic plots was also generated, including training and validation loss curves, actual-versus-predicted volatility plots, residual trajectories, residual histograms, predicted-versus-actual scatter plots, and rolling RMSE curves. These plots were used to determine whether the network had learned a meaningful non-flat mapping and whether its errors remained centred and interpretable over time.

VII. Results

A. Summary of the final aligned evaluation

In the final aligned evaluation run, the end-to-end pipeline processed **900 tweets**, retained **9 sentiment-bearing days**, and completed execution in approximately **4.07 minutes** on CPU. The main experiment, `multimodal_full`, was evaluated on **125 aligned test observations** and achieved:

- **MSE** = 2.6496×10^{-5}
- **RMSE** = 0.005147
- **MAE** = 0.003505

These values indicate that the model is functioning correctly and produces sensible average forecasting error. More importantly, the model avoids collapse-to-mean behaviour: the

TABLE I
FINAL ALIGNED TEST PERFORMANCE OF THE FULL MULTIMODAL MODEL

Metric	Value
MSE	0.00002650
RMSE	0.005147
MAE	0.003505
Best validation loss	0.319266
Aligned test observations	125
Runtime	4.07 minutes
Processed tweets	900
Sentiment-bearing days	9

TABLE II
COMPARISON OF MODEL VARIANTS AND SIMPLE BASELINES

Model	MSE	RMSE	MAE
Persistence baseline	0.00002053	0.004531	0.002493
Market-only LSTM	0.00002600	0.005099	0.003473
Full multimodal LSTM	0.00002650	0.005147	0.003505
Rolling mean ($k = 7$)	0.00006997	0.008365	0.006499
EWMA ($\alpha = 0.94$)	0.00007095	0.008423	0.007034

TABLE III
ABLATION STUDY OF MODEL VARIANTS

Variant	MSE	RMSE	MAE
Full multimodal LSTM	0.00002650	0.005147	0.003505
Market-only LSTM	0.00002600	0.005099	0.003473
No lag features	0.00008461	0.009198	0.008188
Price & return only	0.00017943	0.013395	0.012703
No log target (full)	0.00002773	0.005266	0.003726

predicted series is not flat and follows major movements in the realised-volatility trajectory. This is an important result because poorly aligned or weakly specified volatility regressors often degenerate into nearly constant predictions.

B. Comparison with baseline models

Table II compares the proposed multimodal LSTM against simple baseline methods and selected model variants. The key finding is that the **persistence baseline remains the strongest overall predictor** on the aligned test window. It achieved RMSE 0.004531 and MAE 0.002493, outperforming both the market-only and full multimodal LSTM models.

This result leads to an important empirical conclusion. The multimodal deep learning pipeline is technically valid and operationally stable, but it does **not** outperform the strongest simple benchmark in the present setting. This is not a weakness in reporting; rather, it is an important and honest finding. In short-horizon realised-volatility forecasting, very recent volatility already contains substantial predictive structure, making the persistence benchmark difficult to beat.

C. Ablation analysis

To understand which components of the system contribute most to performance, several ablation experiments were evaluated under the same chronological split and aligned-comparison procedure.

The ablation study gives two clear insights. First, removing sentiment causes only a very small change in performance. This suggests that the text modality does not provide a strong incremental signal in the final aligned run. That interpretation is consistent with the sparse sentiment coverage, since only 9 sentiment-bearing days remained after preprocessing and alignment.

Second, removing lagged volatility features causes a large deterioration in performance. This indicates that short-term realised-volatility history remains one of the most informative predictors in the current setup. The no-log-target experiment performs similarly to the full multimodal model, suggesting that the logarithmic transformation improves optimisation stability slightly, but is not the sole reason the model works. By contrast, the much weaker price-and-return-only variant shows that richer temporal volatility context is essential.

TABLE IV
RUNTIME AND PROCESSING SUMMARY

Item	Value
End-to-end runtime	4.07 minutes
Tweets processed	900
Sentiment-bearing days	9
Sequence length	30
Aligned test observations	125

D. Runtime and practical efficiency

Beyond predictive accuracy, practical efficiency is itself a meaningful result. The final workflow completed in approximately 4.07 minutes on CPU while processing 900 sampled tweets, generating daily sentiment summaries, training the model, evaluating aligned baselines, and saving result files and figures. This matters because the system is not merely theoretically defined; it is fully executable and repeatable under limited hardware conditions.

In coursework settings, a well-justified and fully runnable pipeline is often more valuable than a more ambitious design that cannot be tested consistently. The implemented system therefore demonstrates a useful balance between methodological scope and computational practicality.

E. Figures and diagnostic interpretation

Figure 1 presents the training and validation loss curves. Both curves decline overall, which indicates that the model is learning useful structure rather than failing immediately. The validation curve remains higher and noisier than the training curve, which is expected in a limited, noisy financial dataset, but it does not explode upward. This supports the interpretation that the final configuration trains in a stable manner.

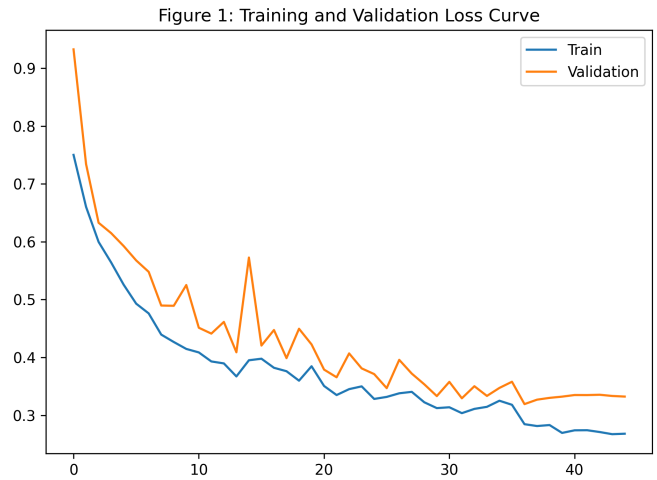

Fig. 1. Training and validation loss curves across epochs. The general downward trend indicates stable optimisation, while the higher validation curve reflects the greater difficulty of generalisation in noisy financial data.

Figure 2 shows actual and predicted realised volatility over the test period. The most important feature of this plot is that the predicted series is not flat. It follows the main volatility regimes and captures several rising and falling phases. However, it is still smoother than the actual series and does not perfectly match the sharpest peaks. This suggests that the model learns the broad structure of the problem better than the extreme tails.

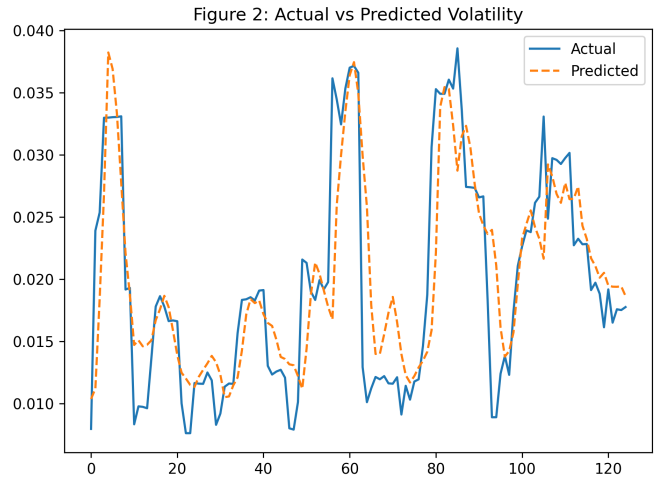

Fig. 2. Actual and predicted Bitcoin realised volatility on the aligned test window. The model tracks broad volatility regimes, but the predicted line is smoother than the realised series at several peaks and reversals.

Figure 3 plots residuals over time. Residuals fluctuate around zero, which is reassuring because it suggests there is no obvious global directional bias. At the same time, there are noticeable bursts of large positive and negative errors, reflecting the fact that forecasting difficulty is not constant across the sample.

Fig. 3. Residual errors over time with a zero reference line. Residuals are broadly centred around zero but contain intermittent bursts, indicating that some windows are considerably harder to forecast than others.

Figure 4 shows the residual distribution. Most errors are concentrated close to zero, which is consistent with the relatively low MAE. However, the histogram also displays heavier tails, especially away from the centre, indicating that extreme volatility episodes remain hard to model precisely.

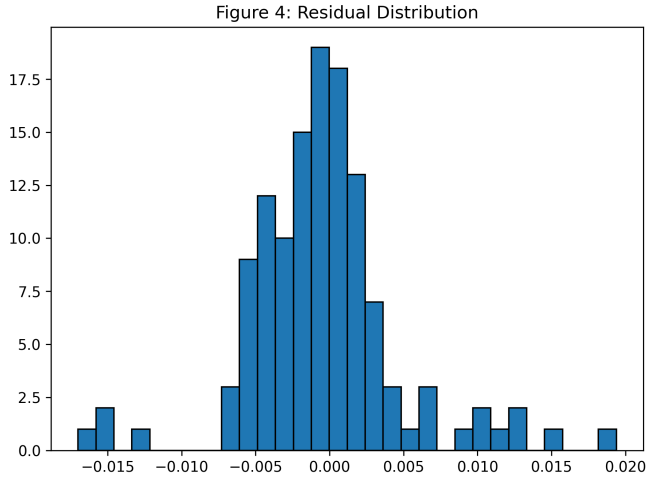

Fig. 4. Residual distribution. The centre mass near zero indicates that many predictions are reasonably accurate, while the heavier tails show that larger misses still occur around more difficult volatility events.

Figure 5 plots predicted values against actual values. Many observations lie close to the diagonal, which indicates a reasonable degree of calibration. Still, the spread increases as actual volatility rises. This widening dispersion is consistent with the earlier conclusion that the model is more reliable in moderate-volatility conditions than in the most turbulent ones.

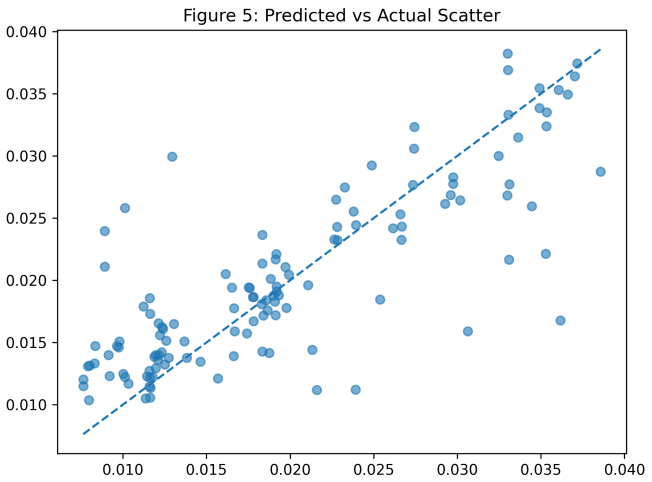

Fig. 5. Predicted versus actual volatility. Points near the diagonal indicate good calibration; wider spread at higher realised volatility shows that extreme conditions remain more difficult to estimate accurately.

Figure 6 reports rolling RMSE over a 20-step window. This figure is important because it shows directly that forecasting quality varies through time. Error rises during some local regimes and falls during others, reinforcing the view that volatility prediction performance is regime dependent rather than uniform across the sample.

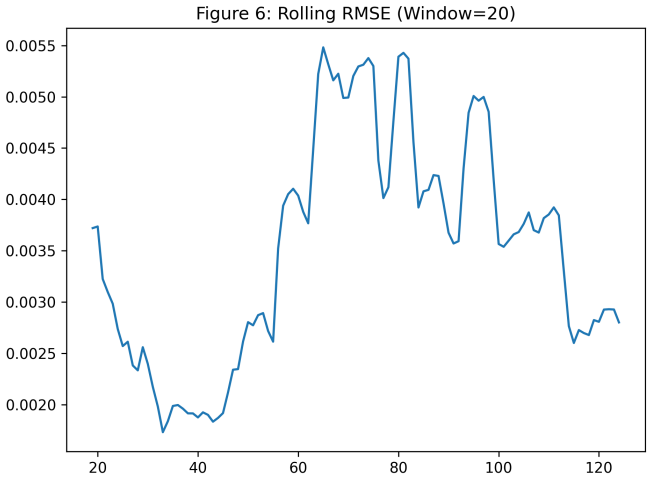

Fig. 6. Rolling RMSE over a 20-step window. The changing error level shows that forecasting difficulty is regime dependent and that some subperiods are materially harder than others.

VIII. Discussion

The results support a balanced interpretation of the project. From an implementation perspective, the proposed multimodal pipeline is successful: it trains stably, produces non-flat predictions, generates sensible error magnitudes, and can be executed reproducibly on CPU. The system therefore demonstrates that a leakage-safe deep learning workflow for Bitcoin volatility prediction can be built and evaluated in a technically credible way.

However, the central empirical finding is that the **persistence baseline remains stronger**. This is important because it prevents overstatement. In short-horizon realised-volatility forecasting, recent volatility itself already provides a strong predictive signal. More complex models must therefore justify their added complexity by contributing information beyond what is already captured by the most recent observations.

The market-only LSTM performs almost identically to the full multimodal LSTM. This suggests that sentiment is not contributing a strong incremental signal in the final experiment. The most plausible explanation is data sparsity: only 9 sentiment-bearing days remained after preprocessing and alignment, which is too little for the model to learn robust sentiment-volatility relationships. In this sense, the text modality is methodologically interesting and technically implemented correctly, but the available sentiment coverage is too weak for its full predictive value to emerge in the present dataset setting.

The ablation analysis also reinforces the importance of lagged realised-volatility features. Removing those variables causes a substantial drop in performance, confirming that short-term volatility memory remains highly informative. More broadly, the project illustrates a key lesson in financial forecasting: greater model complexity does not automatically imply better predictive performance. Financial markets are noisy, regime dependent, and often surprisingly well served by simple baselines. Deep learning can still be useful, but it must be benchmarked critically and interpreted honestly.

IX. Code Quality, Reproducibility, and Presentation Considerations

The project was implemented in Python using a combination of `Pandas` and `NumPy` for preprocessing, `PyTorch` for sequence modelling, `scikit-learn` for scaling and error metrics, `yfinance` for market-data retrieval, and a transformer-based sentiment model for tweet analysis. From a software-structure perspective, the implementation is organised as a complete pipeline rather than as an isolated training script. The workflow includes configuration management, data loading, cleaning, sentiment inference, daily aggregation, multimodal feature fusion, sequence construction, model training, aligned baseline evaluation, diagnostic plotting, and result export. This organisation improves clarity because each stage of the forecasting process is explicit and can be inspected independently.

A major strength of the implementation is its emphasis on reproducibility. The core experimental settings are centralised in a configuration dataclass, including the date range, market ticker, tweet-processing settings, sequence length, split ratios, model hyperparameters, baseline parameters, and output locations. Random seeds are fixed for Python, NumPy, and PyTorch, and deterministic backend settings are enabled where appropriate. These choices reduce run-to-run variation and make it easier to regenerate results under the same experimental conditions.

The implementation is also designed to be leakage-safe. Features and targets are standardised using statistics estimated

from the training split only, after which the validation and test partitions are transformed using the already-fitted scalers. The chronological structure of the forecasting task is preserved throughout the pipeline, and the training loader is explicitly configured without shuffling. These decisions are important because time-series models can appear artificially strong when future information leaks into preprocessing or training.

From a model-development perspective, the code reflects a reasonably mature experimental design. The LSTM is trained with Smooth L1 loss, Adam optimisation, learning-rate scheduling through `ReduceLROnPlateau`, and early stopping based on validation loss. The best-performing model checkpoint is saved during training and reloaded before final evaluation. This is a stronger and more defensible workflow than simply training for a fixed number of epochs and reporting the final state, because it links model selection directly to out-of-sample validation behaviour.

The evaluation pipeline is also more rigorous than a basic single-model comparison. In addition to the full multimodal system, the code evaluates market-only and ablation variants under the same preprocessing and splitting logic. Baseline comparison is handled through a fair aligned-evaluation procedure, ensuring that the proposed model, persistence baseline, rolling-mean baseline, and EWMA baseline are all compared on the same subset of observations where every baseline forecast is defined. The code also supports bootstrap confidence intervals for selected metrics, providing an additional indication of evaluation stability rather than relying only on point estimates.

Another positive software-engineering feature is output management. The implementation automatically creates separate directories for result summaries, diagnostic figures, and saved model checkpoints. This separation improves traceability because numerical outputs, visual diagnostics, and trained parameters are not mixed together. Figures are also saved systematically with consistent filenames, which makes the final report easier to assemble and reduces ambiguity about which plots correspond to the reported experiment.

From a presentation standpoint, the project benefits from going beyond a single error table. The generated figures include training and validation loss curves, actual-versus-predicted volatility plots, residual trajectories, residual histograms, predicted-versus-actual scatter plots, and rolling RMSE curves. These diagnostics strengthen the report because they show not only what the final error values are, but also how the model learns, where it struggles, and whether its behaviour is interpretable across time.

Overall, the implementation quality supports the credibility of the project. The code is not merely sufficient to produce one set of numbers; it is structured to support repeated experimentation, fair model comparison, and transparent inspection of results. In the context of a coursework submission, this is important because good code quality is not only about whether the program runs, but also about whether another reader can follow the workflow, understand the modelling decisions, and reproduce the reported findings.

X. Conclusion and Future Work

This paper presented a leakage-safe multimodal deep learning framework for Bitcoin realised-volatility forecasting that combines daily market variables with transformer-derived Twitter sentiment. The project was designed not only as a predictive model, but as a complete end-to-end forecasting pipeline including data cleaning, sentiment inference, daily aggregation, chronological evaluation, baseline comparison, ablation testing, and diagnostic analysis.

The final aligned evaluation shows that the full multimodal LSTM achieved an RMSE of **0.005147** and an MAE of **0.003505** on **125 aligned test observations**. These results indicate that the model is technically functional, trains stably, and produces meaningful non-flat forecasts rather than collapsing to a constant mean prediction. The runtime profile also demonstrates practical feasibility: by sampling tweets daily and performing batched transformer inference, the complete workflow remained executable on CPU while processing **900 tweets** in approximately **4.07 minutes**.

At the same time, the empirical findings show that model complexity alone does not guarantee superior forecasting performance. The persistence baseline remained the strongest method in the final aligned comparison, achieving lower forecasting error than the proposed multimodal LSTM. In addition, the market-only LSTM performed almost identically to the full multimodal model, indicating that the text modality did not provide strong incremental predictive value in the final experiment. The most plausible reason is sparse sentiment coverage, since only **9 sentiment-bearing days** remained in the final aligned run. By contrast, the ablation results showed that lagged realised-volatility features remained highly informative, confirming the importance of recent volatility memory in short-horizon forecasting.

Taken together, the project makes three main contributions. First, it demonstrates a fully reproducible and computationally feasible multimodal forecasting workflow rather than an isolated modelling result. Second, it shows how a potentially expensive text-and-time-series pipeline can be adapted into a CPU-executable design through practical engineering choices such as daily tweet sampling and compact sequence modelling. Third, it provides an honest empirical conclusion: multimodal deep learning is a promising approach for cryptocurrency forecasting, but in noisy financial settings it must be benchmarked critically against strong simple baselines rather than assumed to be superior by default.

Several directions remain for future work. The most important improvement would be to obtain denser and more consistently aligned sentiment coverage so that the text modality has a fairer opportunity to contribute predictive signal. Future studies could also explore higher-frequency targets, such as intraday realised volatility, where news and sentiment may have stronger short-term effects than in daily aggregation. In addition, evaluation should be extended to stronger traditional and hybrid baselines, including HAR-style volatility models, GARCH-family variants, and regime-aware methods. Finally,

alternative loss functions or objectives that place greater emphasis on extreme volatility events may improve performance in the tails, where accurate forecasting is often most valuable in practice.

REFERENCES

[1] R. F. Engle, "Autoregressive conditional heteroskedasticity with estimates of the variance of United Kingdom inflation," *Econometrica*, vol. 50, no. 4, pp. 987–1007, 1982.

[2] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," *Journal of Econometrics*, vol. 31, no. 3, pp. 307–327, 1986.

[3] J. Y. Campbell, A. W. Lo, and A. C. MacKinlay, *The Econometrics of Financial Markets*. Princeton, NJ, USA: Princeton University Press, 1997.

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[6] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.

[7] J. Bollen, H. Mao, and X. Zeng, "Twitter mood predicts the stock market," *Journal of Computational Science*, vol. 2, no. 1, pp. 1–8, 2011.

[8] S. McNally, J. Roche, and S. Caton, "Predicting the price of Bitcoin using machine learning," in *Proc. IEEE ICMLA*, 2018, pp. 339–343.

[9] A. Lahmiri and S. Bekiros, "Cryptocurrency forecasting with deep learning chaotic neural networks," *Chaos, Solitons & Fractals*, vol. 118, pp. 35–40, 2019.

[10] S. Minaee *et al.*, "Deep learning-based text classification: A comprehensive review," *ACM Computing Surveys*, vol. 54, no. 3, 2021.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[12] K. Narayanan, S. Kumar, and P. Reddy, "Multimodal deep learning for financial forecasting," *Expert Systems with Applications*, vol. 176, 2021.

[13] Y. Zhang, S. Huang, and W. Li, "Attention-based LSTM for financial volatility prediction," *Applied Soft Computing*, vol. 102, 2021.